

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

КУРСОВАЯ РАБОТА

по дисциплине «Обучение с подкреплением»

Тема: Реализация и сравнение разных версий алгоритма Deep Q-Network

Студент гр. 0310

Аксенов И.В.

Студент гр. 0310

Хвостунов М.М.

Студентка гр. 0310

Шкода М.А.

Преподаватель

Глазунов С.А.

Санкт-Петербург

2025

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студенты Аксенов И.В., Хвостунов М.М., Шкода М.А.

Группа 0310

Тема работы: Реализация и сравнение разных версий алгоритма Deep Q-Network

Исходные данные:

Необходимо реализовать и сравнить между собой следующие версии DQN:

- Double Q-learning
- Prioritized replay
- Dueling networks

В качестве окружения для тестирования:

- LunarLander-v3
- Mountain-Car

Предполагаемый объем пояснительной записки:

Не менее 10 страниц

Дата выдачи задания: 11.02.2025

Дата сдачи реферата: 18.05.2025

Дата защиты реферата: 18.05.2025

Студент гр. 0310

Аксенов И.В.

Студент гр. 0310

Хвостунов М.М.

Студентка гр. 0310

Шкода М.А.

Преподаватель

Глазунов С.А.

СОДЕРЖАНИЕ

Введение	4
Роли	5
Постановка задачи	6
1. Среды обучения	6
1.1. Mountain Car	6
1.2. Lunar Lander	7
Сравнение алгоритмов	9
1. Алгоритм DQN	9
2. Алгоритм Double Q-Learning	10
3. Алгоритм Prioritized replay	11
4. Алгоритм Dueling Networks	12
Выводы	13
Приложение А	15

ВВЕДЕНИЕ

В настоящее время развитие нейронных сетей происходит стремительно. Как следствие, они внедряются в различные сферы нашей жизни, начиная с поиска информации, заканчивая получением рекомендаций по различным вопросам через различные модели нейронных сетей. Чтобы создать подобную модель, ее необходимо обучить. Для этого были созданы различные алгоритмы обучения, в частности алгоритмы обучения с подкреплением, среди которых находится алгоритм DQN (Deep Q-Learning). Как DQN решил проблему масштабируемости и стабильности обычного Q-Learning, так и для DQN создаются улучшенные версии алгоритма, чтобы получить более точные результаты. В данной работе будут рассмотрены следующие улучшения алгоритма DQN: Double Q-learning, Prioritized replay, Dueling Networks на примере сред LunarLander-v3 и Mountain-Car.

РОЛИ

Данную работу выполняли студенты группы 0310:

1. Аксенов И.В. занимался организацией кода и пунктом №1 задания (Double Q-learning). Было сделано следующее:
 - Организовал базу для проекта на основе лабораторной работы №1 (https://github.com/moevm/RL_2025/tree/task1_0310_Aksenov);
 - Адаптировал код лабораторной работы под новые среды (Mountain-Car и LunarLander-v3): нормализация состояния, расчет наград и др.;
 - Метод `_alg_double_q` в классе `DQNAgent`;
2. Шкода М.А. занималась пунктом №2 задания (Prioritized Replay). Было сделано следующее:
 - Класс `PrioritizedReplayBuffer`;
 - Добавила вариант алгоритма `prioritized` в класс `DQNAgent` для использования буфера `PrioritizedReplayBuffer`.
 - Логика обучения в методе `_train_prioritized_replay_buffer` в классе `DQNAgent`;
 - Расчет потерь в методе `_alg_prioritized` в классе `DQNAgent`.
3. Хвостунов М.М. занимался пунктом №3 задания (Dueling Networks). Было сделано следующее:
 - Функция `dueling_experiment` для проведения эксперимента;
 - Класс нейронной сети `DuelingQNetwork`;
 - Добавил параметр `dueling` в класс `DQNAgent` для проведения обучения на основе `DuelingQNetwork`.

ПОСТАНОВКА ЗАДАЧИ

Каждый шаг времени t описывается следующим набором параметров:

- S_t - состояние среды в момент времени t ;
- A_t - действие, которое принимает агент в момент времени t ;
- γ_{t+1} - коэффициент дисконтирования;
- S_{t+1} - новое состояние после применения действия A_t ;

Алгоритм Deep Q-Learning работает на основе следующей формулы:

$$\left(R_{t+1} + \gamma_{t+1} \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t) \right)^2 \quad (1)$$

Она состоит из двух сетей:

- Online network - которая является основной и выбирает следующие действия на основе состояния в данный момент времени;
- Target network - копия основной сети, которая обновляется реже с целью получения более устойчивой модели. В формуле она описывается значениями $\bar{\theta}$

1. Среды обучения

В качестве сред для обучения будут использоваться LunarLander-v3 и Mountain-Car.

1.1. Mountain Car

Визуальное представление среды представлено на рисунке 1.

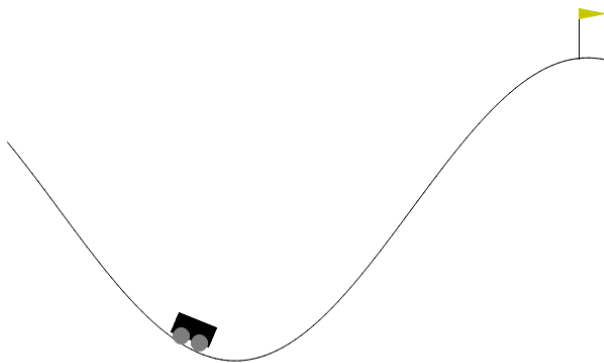


Рисунок 1 – Вид среды Mountain Car

Пространство наблюдений для данной среды представлено в таблице 1.

Таблица 1 – Пространство наблюдений

Num	Observation	Min	Max	Unit
0	position of the car along the x-axis	-1.2	0.6	position (m)
1	velocity of the car	-0.07	0.07	velocity (v)

Пространство действий является дискретным и состоит из следующих значений:

- 0: Ускорение в левую сторону
- 1: Не ускоряться никуда
- 2: Ускоряться в правую сторону

Целью модели является достижение флага, расположенного на вершине правого холма.

В данной работе использовались значения среды, которые представлены в документации gymnasium:

```
env = gym.make(
    "MountainCar-v0",
    render_mode="rgb_array",
    goal_velocity=0.1
)
```

1.2. Lunar Lander

Визуальное представление среды представлено на рисунке 2.

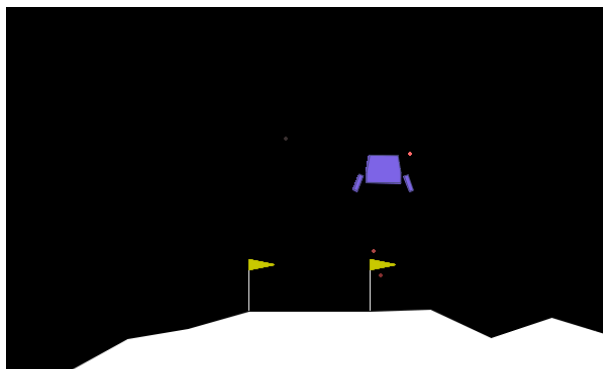


Рисунок 2 – Вид среды Lunar Lander

Пространство наблюдений в данном случае это вектор из 8 значений:

- 2 значения позиции x, y ;
- 2 значения скорости x, y ;

- угол наклона корабля;
- угловая скорость;
- 2 булевых значения, которые отображают наличие контакта ног корабля с поверхностью;

Пространство действий состоит из 4 возможных значений:

- 0: Ничего не делать
- 1: Использовать левый двигатель
- 2: Использовать основной двигатель
- 3: Использовать правый двигатель

Награда дается на каждом шаге. Общая награда - сумма всех наград в ходе работы.

С каждым шагом награда:

- увеличивается/уменьшается, чем ближе/дальше корабль находится от места посадки;
- увеличивается/уменьшается, чем медленнее/быстрее корабль двигается;
- уменьшается в зависимости от угла наклона корабля (если угол не перпендикулярен горизонту);
- увеличивается на 10 очко за каждую ногу корабля, контактирующую с поверхностью;
- уменьшается на 0.03 очка за каждый кадр, когда корабль использует боковые двигатели;
- уменьшается на 0.3 очка за каждый кадр, когда корабль использует основной двигатель.

За эпизод дается +100 или -100 очков, в зависимости от того, приземлился корабль успешно и наоборот.

Эпизод с наградой больше или равной 200 будет являться решением.

В данной работе использовались значения среды, которые представлены в документации gymnasium:

```
env = gym.make(
    "LunarLander-v3",
```



```

continuous=False,
gravity=-10.0,
enable_wind=False,
wind_power=15.0,
turbulence_power=1.5,

```

)

СРАВНЕНИЕ АЛГОРИТМОВ

1. Алгоритм DQN

На рисунке 3 представлен график наград и потерь для среды Lunar Lander.

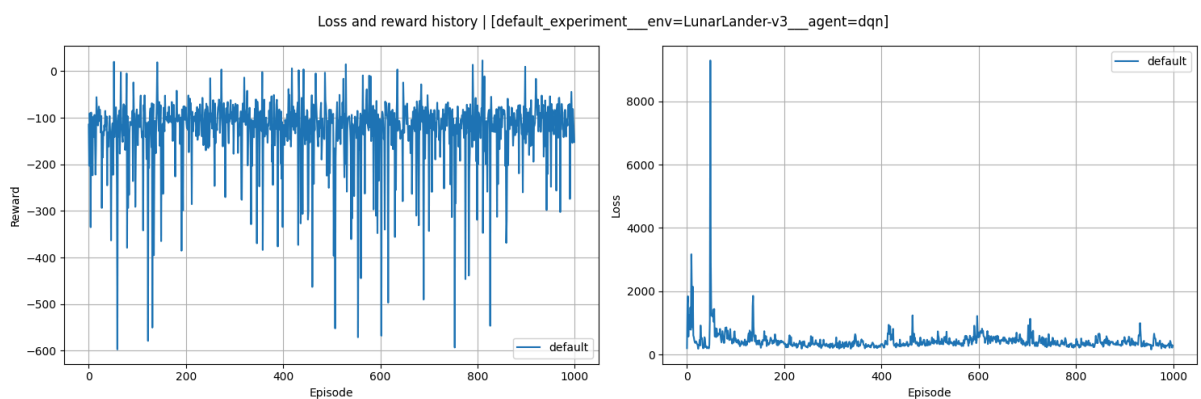


Рисунок 3 – График потерь для среды Lunar Lander

На рисунке 4 представлен график наград и потерь для среды Mountain Car.

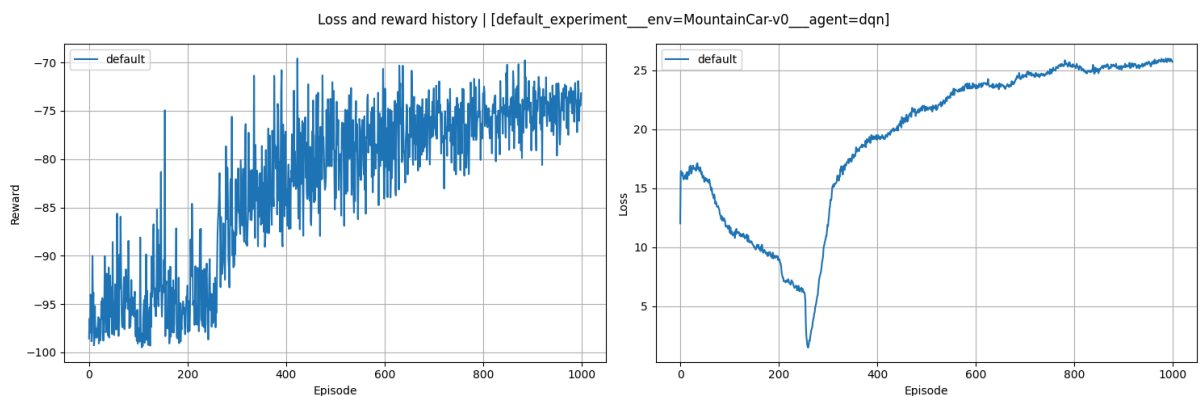


Рисунок 4 – График потерь для среды Mountain Car

По получившимся результатам можно сделать следующие выводы:

- Lunar Lander:
 - Рост количества эпизодов уменьшает долю потерь;

- Сильного изменения в размере награды не наблюдается. Это может возникать из-за установленных гиперпараметров.
- MountainCar:
 - Увеличение количества эпизодов ведет к увеличению размера награды, однако, размер потерь также возрастает.

2. Алгоритм Double Q-Learning

Данный алгоритм работает по следующей формуле:

$$\left(R_{t+1} + \gamma_{t+1} q_{\bar{\theta}} \left(S_{t+1}, \operatorname{argmax}_{a'} q_{\theta}(S_{t+1}, a') \right) - q_{\theta}(S_t, A_t) \right)^2 \quad (2)$$

На рисунке 5 представлен график наград и потерь для среды Lunar Lander.

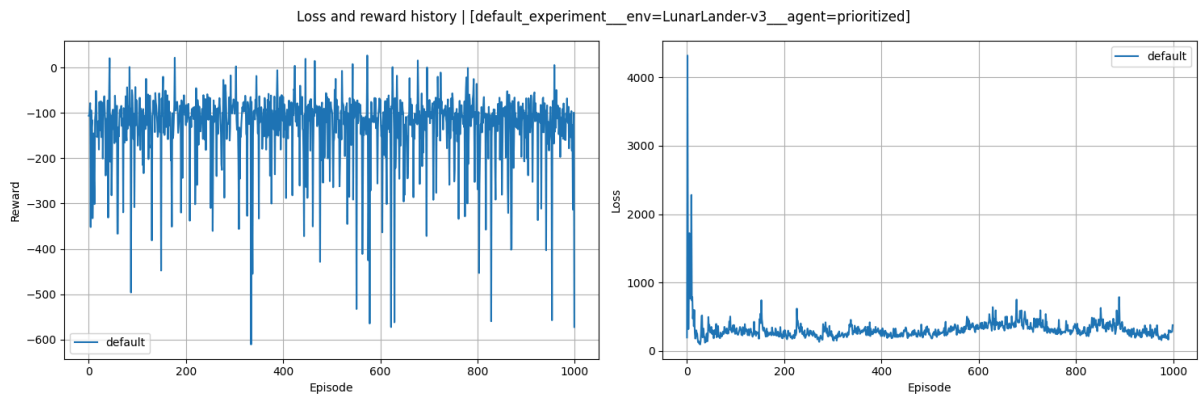


Рисунок 5 – График потерь для среды Lunar Lander

На рисунке 6 представлен график наград и потерь для среды Mountain Car.

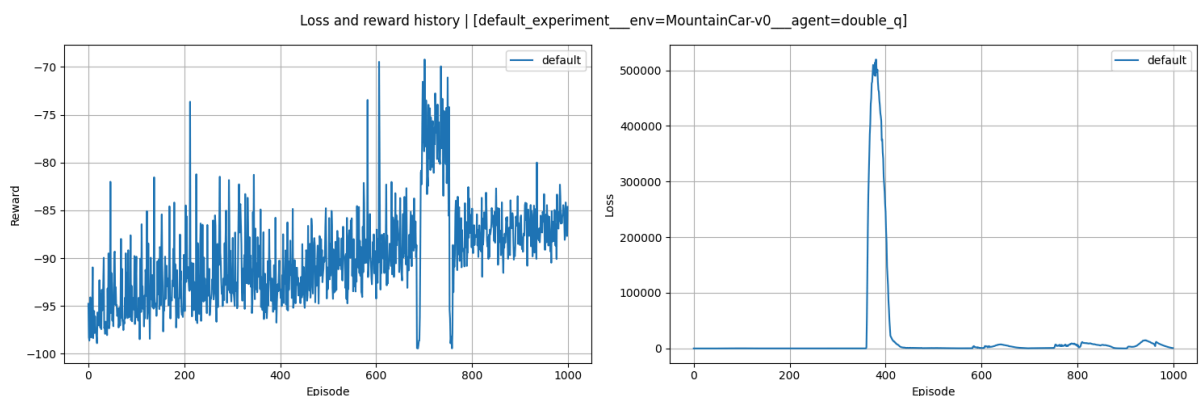


Рисунок 6 – График потерь для среды Mountain Car

По получившимся результатам можно сделать следующие выводы:

- Lunar Lander:
 - Получение наград получается примерно одинаковым;

- Размер потерь с ростом количества эпизодов уменьшается.
- MountainCar:
 - Можно заметить, в целом, положительный рост размера награды с увеличением количества эпизодов;
 - Потери ведут себя стабильно, только есть область с неким “выбросом”.

3. Алгоритм Prioritized replay

В обычном replay все переходы являются равновероятными. В случае Prioritized replay вероятность переходов определяется TD-ошибкой, которая вычисляется по следующей формуле:

$$p_t \propto | R_{t+1} + \gamma_{t+1} \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t) |^{\omega} \quad (3)$$

Где ω - гиперпараметр, который определяет форму распределения.

На рисунке 7 представлен график наград и потерь для среды Lunar Lander.

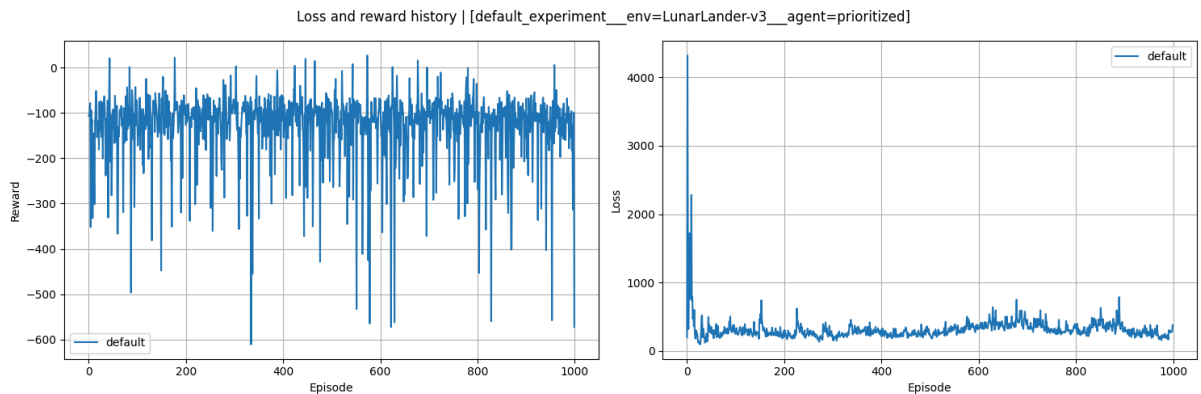


Рисунок 7 – График потерь для среды Lunar Lander

На рисунке 8 представлен график наград и потерь для среды Mountain Car.

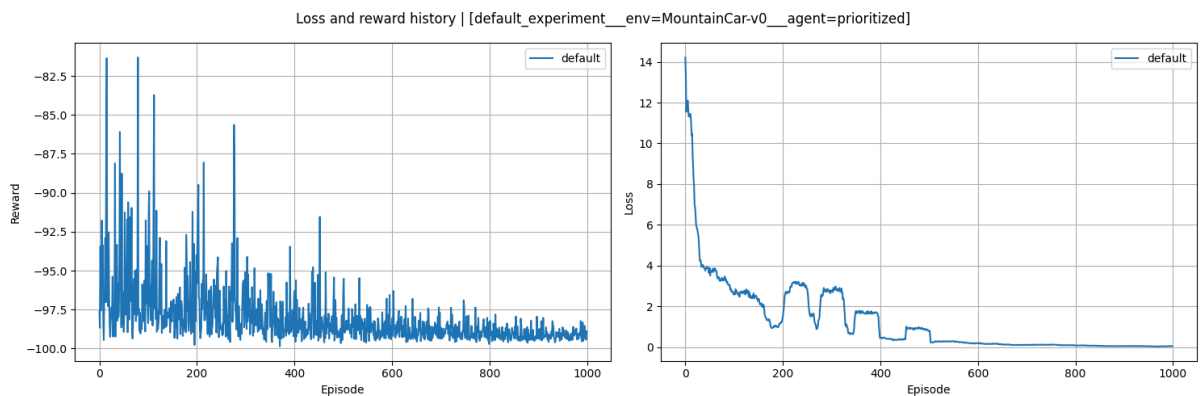


Рисунок 8 – График потерь для среды Mountain Car

По получившимся результатам можно сделать следующие выводы:

- Lunar Lander:
 - Награда на протяжении всех эпизодов ведет себя по большей части одинаково
 - Количество потерь при росте количества эпизодов уменьшается
- MountainCar:
 - Потери и награды с ростом количества эпизодов уменьшаются.

4. Алгоритм Dueling Networks

Суть алгоритма заключается в двух потоках обработки, потока значений (values) и потока преимуществ (advantages), общего сверточного кодировщика и собранные специальным агрегатором. Для расчета q-значений используется следующая формула:

$$q_{\theta}(s, a) = v_{\eta}(f_{\xi}(s)) + a_{\psi}(f_{\xi}(s), a) - \frac{\sum_{a'} a_{\psi}(f_{\xi}(s), a')}{N_{\text{actions}}} \quad (4)$$

Где ξ - параметр общего кодировщика f_{ξ} , η - параметр значений v_{η} , ψ - параметр преимуществ a_{ψ} .

На рисунке 9 представлен график наград и потерь для среды Lunar Lander.

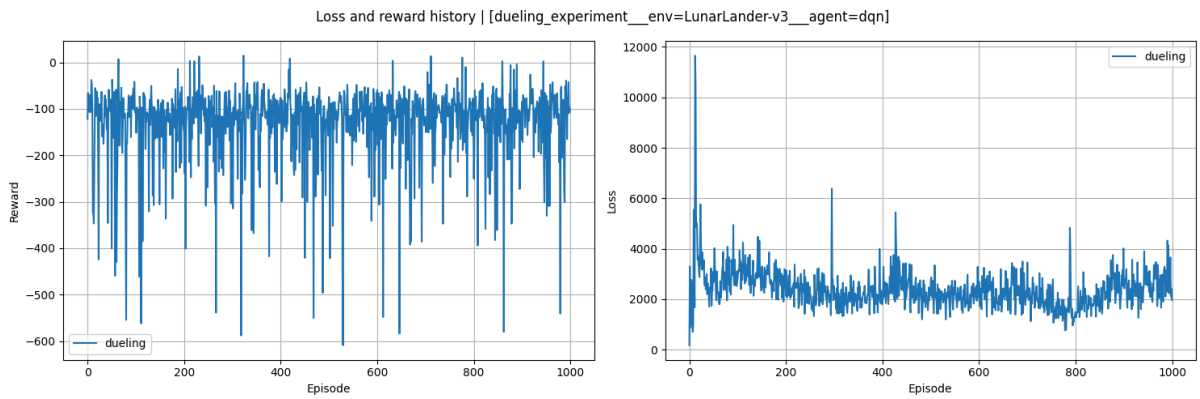


Рисунок 9 – График потерь для среды Lunar Lander

На рисунке 10 представлен график наград и потерь для среды Mountain Car.

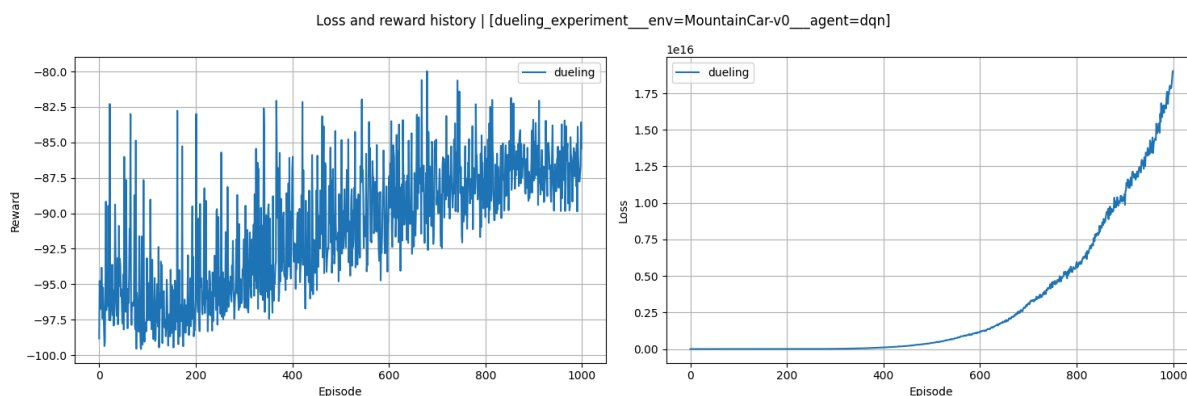


Рисунок 10 – График потерь для среды Mountain Car

По получившимся результатам можно сделать следующие выводы:

- Lunar Lander:
 - Награды и потери ведут себя достаточно стабильно и с ростом количества эпизодов результаты сильно не меняются, поэтому можно предположить, что улучшение модели будет не сильное при увеличении количества эпизодов.
- MountainCar:
 - Рост потерь может говорить о том, что в перспективе результаты могут ухудшаться, потому что размер потерь также увеличивается с ростом количества эпизодов;

ВЫВОДЫ

В ходе данной работы были рассмотрены улучшения алгоритма DQN - Double Q-learning, Prioritized replay, Dueling Networks. Они были рассмотрены на средах:

- LunarLander:
 - Обычный DQN и все его улучшения в целом дали схожие результаты, различался лишь размер потерь. В алгоритме Dueling Networks потери вели себя наименее стабильно, чем во всех остальных случаях.
- MountainCar:
 - Лучший результат по количеству наград выдал обычный алгоритм DQN, однако размер потерь также имел тенденцию роста, что может

говорить о том, что при увеличении количества эпизодов результат может ухудшиться;

- Следующий лучший результат получился при алгоритме Double Q-learning. Был участок, который можно посчитать выбросом. В остальном рост количества эпизодов в целом вел к увеличению размера награды, а размер потери вели себя достаточно стабильно;
- Далее идет алгоритм Dueling Networks. Размер награды увеличивался по мере увеличения количества эпизодов, однако размер потерь также увеличивается и график напоминает экспоненциальную функцию, на основе чего можно сделать вывод, что при дальнейшем росте количества эпизодов размер потерь будет очень сильно увеличиваться;
- Хуже всего себя показал Prioritized Replay. За все эпизоды количество наград лишь уменьшалось и приближалось к отметке -100 . На основе этого можно сделать вывод, что данный алгоритм является самым неудачным выбором при работе с Mountain Car.

ПРИЛОЖЕНИЕ А

Исходный код

```
import csv
import os
import random
from collections import deque
from datetime import datetime

import gymnasium as gym
import matplotlib.pyplot as plt
import numpy as np
import torch
from gymnasium.core import Env
from torch import nn, optim
from tqdm import tqdm

seed = 310
np.random.seed(seed)
np.random.default_rng(seed)
os.environ["PYTHONHASHSEED"] = str(seed)
torch.manual_seed(seed)
if torch.cuda.is_available():
    torch.cuda.manual_seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

def plot_reward_and_loss(sim_results, name, width=6, height=6, show_fig=False):
    fig, ax = plt.subplots(1, 2, figsize=(width, height))
    fig.suptitle(f"Loss and reward history | [{name}]")

    for sim_name, sim_results in sim_results.items():
        print("plotting: ", sim_name)

        ax[0].plot(sim_results["reward"], label=f"{sim_name}")
        ax[0].set_xlabel("Episode")
        ax[0].set_ylabel("Reward")

        ax[1].plot(sim_results["loss"], label=f"{sim_name}")
        ax[1].set_xlabel("Episode")
        ax[1].set_ylabel("Loss")

    ax[0].legend()
    ax[0].grid()
    ax[1].legend()
    ax[1].grid()

    fig.tight_layout()
    os.makedirs("fig", exist_ok=True)
    fig.savefig(f"fig/{name}_loss_and_reward_history.png")

    if show_fig:
        plt.show()

def plot_fig_single(sim_results, name, width=6, height=6, show_fig=False):
    fig1, ax1 = plt.subplots(figsize=(width, height))
    fig2, ax2 = plt.subplots(figsize=(width, height))
    fig1.suptitle("История наград")
    fig2.suptitle("История потерь")
```

```

for sim_name, sim_results in sim_results.items():
    print("plotting: ", sim_name)

    ax1.plot(sim_results["reward"], label=f"{sim_name}")
    ax1.set_xlabel("Episode")
    ax1.set_ylabel("Reward")

    ax2.plot(sim_results["loss"], label=f"{sim_name}")
    ax2.set_xlabel("Episode")
    ax2.set_ylabel("Loss")

    ax1.legend()
    ax1.grid()
    ax2.legend()
    ax2.grid()

    fig1.tight_layout()
    fig2.tight_layout()
    os.makedirs("fig", exist_ok=True)
    fig1.savefig(f"fig/{name}_reward_history.png")
    fig2.savefig(f"fig/{name}_loss_history.png")

    if show_fig:
        plt.show()

def dueling_experiment(
    env, obs_size, n_actions, params, env_name: str, agent_algorithm: str =
"default"
):
    simulation_results = {}

    print("Dueling experiment")
    agent = DQNAgent(
        obs_size=obs_size,
        n_actions=n_actions,
        layers=params["layers"](obs_size, n_actions),
        gamma=params["gamma"],
        epsilon=params["epsilon"],
        epsilon_decay=params["epsilon_decay"],
        epsilon_min=params["epsilon_min"],
        batch_size=params["batch_size"],
        learning_rate=params["lr"],
        algorithm=agent_algorithm,
        dueling=True,
    )

    simulation = Simulation(
        env=env,
        agent=agent,
        sim_name="dueling_experiment",
        time=datetime.now(),
        num_steps=params["num_steps"],
        num_episodes=params["num_episodes"],
    )

    simulation.run(env_name)
    simulation.save_to_csv(
        f"dueling_default_experiment__env={env_name}
__agent={agent_algorithm}.csv"
    )

```



```

simulation_results["dueling"] = {
    "reward": simulation.reward_history,
    "loss": simulation.loss_history,
}

plot_reward_and_loss(
    simulation_results,
    name=f"dueling_experiment__env={env_name}__agent={agent_algorithm}",
    width=15,
    height=5,
)

def default_experiment(
    env, obs_size, n_actions, params, env_name: str, agent_algorithm: str = "dqn"
):
    simulation_results = {}

    print("Default experiment")
    agent = DQNAgent(
        obs_size=obs_size,
        n_actions=n_actions,
        layers=params["layers"](obs_size, n_actions),
        gamma=params["gamma"],
        epsilon=params["epsilon"],
        epsilon_decay=params["epsilon_decay"],
        epsilon_min=params["epsilon_min"],
        batch_size=params["batch_size"],
        learning_rate=params["lr"],
        algorithm=agent_algorithm,
    )

    simulation = Simulation(
        env=env,
        agent=agent,
        sim_name="epsilon_experiment",
        time=datetime.now(),
        num_steps=params["num_steps"],
        num_episodes=params["num_episodes"],
        modify_reward=params["modify_rewards"],
    )

    simulation.run(env_name)
    simulation.save_to_csv(
        f"default_experiment__env={env_name}__agent={agent_algorithm}.csv"
    )

    simulation_results["default"] = {
        "reward": simulation.reward_history,
        "loss": simulation.loss_history,
    }

    plot_reward_and_loss(
        simulation_results,
        name=f"default_experiment__env={env_name}__agent={agent_algorithm}",
        width=15,
        height=5,
    )

class ReplayBuffer:
    def __init__(self, capacity=1000):
        self.buffer = deque(maxlen=capacity)

```

```

def push(self, state, action, reward, next_state, done):
    self.buffer.append((state, action, reward, next_state, done))

def sample(self, batch_size):
    batch = random.sample(self.buffer, batch_size)
    states, actions, rewards, next_states, dones = zip(*batch)
    return (
        torch.tensor(np.array(states), dtype=torch.float32),
        torch.tensor(np.array(actions), dtype=torch.float32),
        torch.tensor(np.array(rewards), dtype=torch.float32),
        torch.tensor(np.array(next_states), dtype=torch.float32),
        torch.tensor(np.array(dones), dtype=torch.float32),
    )

def __len__(self):
    return len(self.buffer)

class PrioritizedReplayBuffer:
    def __init__(self, capacity=1000, alpha=0.6):
        self.capacity = capacity
        self.alpha = alpha
        self.buffer = []
        self.priorities = []
        self.next_idx = 0

    def push(self, state, action, reward, next_state, done):
        max_priority = max(self.priorities, default=1.0)

        if len(self.buffer) < self.capacity:
            self.buffer.append((state, action, reward, next_state, done))
            self.priorities.append(max_priority)
        else:
            self.buffer[self.next_idx] = (state, action, reward, next_state, done)
            self.priorities[self.next_idx] = max_priority
            self.next_idx = (self.next_idx + 1) % self.capacity

    def sample(self, batch_size, beta=0.4):
        if len(self.buffer) == 0:
            raise ValueError("Buffer is empty")

        priorities = np.array(self.priorities)[: len(self.buffer)]
        probs = priorities**self.alpha
        probs /= probs.sum()

        indices = np.random.choice(len(self.buffer), batch_size, p=probs)
        samples = [self.buffer[idx] for idx in indices]

        total = len(self.buffer)
        weights = (total * probs[indices])** (-beta)
        weights /= weights.max()
        weights = torch.tensor(weights, dtype=torch.float32)

        states, actions, rewards, next_states, dones = zip(*samples)
        return (
            torch.tensor(np.array(states), dtype=torch.float32),
            torch.tensor(np.array(actions), dtype=torch.float32),
            torch.tensor(np.array(rewards), dtype=torch.float32),
            torch.tensor(np.array(next_states), dtype=torch.float32),
            torch.tensor(np.array(dones), dtype=torch.float32),
            torch.tensor(indices),
            weights,

```

```

    )

    def update_priorities(self, indices, td_errors, epsilon=1e-6):
        for idx, td_error in zip(indices, td_errors):
            self.priorities[idx] = abs(td_error.item()) + epsilon

    def __len__(self):
        return len(self.buffer)

class QNetwork(nn.Module):
    def __init__(self, obs_size, n_actions, layers: list[nn.Module]):
        super(QNetwork, self).__init__()
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

class DuelingQNetwork(nn.Module):
    def __init__(self, obs_size, n_actions):
        super().__init__()
        self.feature = nn.Sequential(
            nn.Linear(obs_size, 128),
            nn.ReLU(),
        )
        self.value_stream = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
        )
        self.advantage_stream = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, n_actions),
        )

    def forward(self, x):
        x = self.feature(x)
        value = self.value_stream(x)
        advantage = self.advantage_stream(x)
        return value + advantage - advantage.mean(dim=1, keepdim=True)

class DQNAgent:
    def __init__(
        self,
        obs_size: int,
        n_actions: int,
        layers: list[nn.Module],
        gamma: float = 0.99,
        epsilon: float = 1.0,
        epsilon_decay: float = 0.955,
        epsilon_min: float = 0.01,
        batch_size: int = 64,
        learning_rate: float = 0.001,
        algorithm: str = "dqn", # or double_q or prioritized
        dueling: bool = False,
    ):
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.dueling = dueling

        if self.dueling:

```

```

        self.q_net = DuelingQNetwork(obs_size, n_actions).to(self.device)
        self.net_target = DuelingQNetwork(obs_size, n_actions).to(self.device)
    else:
        self.q_net = QNetwork(obs_size, n_actions, layers).to(self.device)
        self.net_target = QNetwork(obs_size, n_actions, layers).to(self.device)

    self.net_target.load_state_dict(self.q_net.state_dict())
    self.optimizer = optim.Adam(self.q_net.parameters(), lr=learning_rate)

    self.obs_size = obs_size
    self.n_actions = n_actions
    self.layers = layers

    self.gamma = gamma
    self.epsilon = epsilon
    self.epsilon_decay = epsilon_decay
    self.epsilon_min = epsilon_min
    self.batch_size = batch_size
    self.algorithm = algorithm

    if self.algorithm == "prioritized":
        self.replay_buffer = PrioritizedReplayBuffer(10000)
    else:
        self.replay_buffer = ReplayBuffer(10000)

    self.loss: float = 0.0

def select_action(self, state):
    if random.random() < self.epsilon:
        return random.randint(0, self.n_actions - 1)

    with torch.no_grad():
        state_tensor = torch.tensor(
            state, dtype=torch.float32, device=self.device
        ).unsqueeze(0)
        q_values = self.q_net(state_tensor)

        return torch.argmax(q_values).item()

def _train_prioritized_replay_buffer(self):
    if not (isinstance(self.replay_buffer, PrioritizedReplayBuffer)):
        raise ValueError(
            f"Prioritized algorithm requires PrioritizedReplayBuffer. Current
is: {type(PrioritizedReplayBuffer)}"
        )

    states, actions, rewards, next_states, dones, indices, weights = (
        self.replay_buffer.sample(self.batch_size)
    )
    states = states.to(self.device)
    actions = actions.to(self.device).long()
    rewards = rewards.to(self.device)
    next_states = next_states.to(self.device)
    dones = dones.to(self.device)
    indices = indices.to(self.device)
    weights = weights.to(self.device)

    loss = self._alg_prioritized(
        states,
        actions,
        rewards,
        next_states,
        dones,

```

```

        indices,
        weights,
    )

    return loss

def _train_default_replay_buffer(self):
    if not (isinstance(self.replay_buffer, ReplayBuffer)):
        raise ValueError(
            f"DQN or Double DQN algorithm requires standard ReplayBuffer.
Current is: {type(PrioritizedReplayBuffer)}"
        )

    states, actions, rewards, next_states, dones = self.replay_buffer.sample(
        self.batch_size
    )

    states = states.to(self.device)
    actions = actions.to(self.device).long()
    rewards = rewards.to(self.device)
    next_states = next_states.to(self.device)
    dones = dones.to(self.device)

    if self.algorithm == "dqn":
        loss = self._alg_dqn(
            states,
            actions,
            rewards,
            next_states,
            dones,
        )
    elif self.algorithm == "double_q":
        loss = self._alg_double_q(
            states,
            actions,
            rewards,
            next_states,
            dones,
        )
    else:
        raise ValueError(f"Unknown algorithm: {self.algorithm}")

    return loss

def train(self):
    if len(self.replay_buffer) < self.batch_size:
        return

    if self.algorithm == "prioritized":
        loss = self._train_prioritized_replay_buffer()
    else:
        loss = self._train_default_replay_buffer()

    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    self.loss = loss.item()

def update_epsilon(self):
    self.epsilon = max(self.epsilon * self.epsilon_decay, self.epsilon_min)

def _alg_dqn(

```

```

        self,
        states,
        actions,
        rewards,
        next_states,
        dones,
    ):
        q_values = self.q_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)

        next_q_values = self.net_target(next_states).max(1)[0]
        target_q_values = rewards + self.gamma * next_q_values * (1 - dones)

        loss = nn.MSELoss()(q_values, target_q_values)
        return loss

    def _alg_double_q(
        self,
        states,
        actions,
        rewards,
        next_states,
        dones,
    ):
        next_q_values_online = self.q_net(next_states)
        next_actions = torch.argmax(next_q_values_online, dim=1)

        next_q_values_target = self.net_target(next_states)
        next_q_values = next_q_values_target.gather(
            1, next_actions.unsqueeze(1)
        ).squeeze(1)

        target_q_values = rewards + self.gamma * next_q_values * (1 - dones)

        current_q_values = self.q_net(states).gather(1,
actions.unsqueeze(1)).squeeze(1)

        loss = nn.MSELoss()(current_q_values, target_q_values.detach())
        return loss

    def _alg_prioritized(
        self,
        states,
        actions,
        rewards,
        next_states,
        dones,
        indices,
        weights,
    ):
        if not (isinstance(self.replay_buffer, PrioritizedReplayBuffer)):
            raise ValueError(
                f"Prioritized algorithm requires PrioritizedReplayBuffer. Current
is: {type(PrioritizedReplayBuffer)}"
            )

        current_q_values = self.q_net(states).gather(1,
actions.unsqueeze(1)).squeeze(1)
        next_q_values = self.net_target(next_states).max(1)[0]
        target_q_values = rewards + self.gamma * next_q_values * (1 - dones)
        td_errors = target_q_values - current_q_values

        loss = (weights * td_errors.pow(2)).mean()
        self.replay_buffer.update_priorities(indices, td_errors.detach())

```

```

        return loss

    def update_target(self):
        self.net_target.load_state_dict(self.q_net.state_dict())

class Simulation:
    def __init__(
        self,
        env: Env,
        agent: DQNAgent,
        time: datetime,
        sim_name: str = "simulation",
        num_episodes: int = 1000,
        num_steps: int = 200,
        update_target_frequency: int = 1,
        modify_reward: bool = False,
        seed: int = 310,
    ):
        self.env = env

        self.agent = agent
        self.sim_name = sim_name
        self.time = time
        self.seed = seed

        self.num_episodes = num_episodes
        self.num_steps = num_steps
        self.update_target_frequency = update_target_frequency
        self.modify_reward = modify_reward

        self.reward_history = []
        self.loss_history = []

    def save_to_csv(self, filename: str):
        os.makedirs("csv", exist_ok=True)
        filepath = os.path.join("csv", filename)

        with open(filepath, mode="w", newline="") as file:
            writer = csv.writer(file)
            writer.writerow(
                [
                    "Episode",
                    "Reward",
                    "Loss",
                ]
            )

            for episode, (reward, loss) in enumerate(
                zip(self.reward_history, self.loss_history), start=1
            ):
                writer.writerow([episode, reward, loss])

        print(f"=== Saving csv to {filepath}")

    def normalize_state(self, state, env):
        low = env.observation_space.low
        high = env.observation_space.high
        return (state - low) / (high - low)

    def modify_reward(self, state):
        pos, vel = state

```

```

vel = np.interp(vel, np.array([0, 1]), np.array([-0.5, 0.5]))

degree = pos * 360
degree2radian = np.deg2rad(degree)
reward = 0.2 * (np.cos(degree2radian) + 2 * np.abs(vel))

reward -= 0.5

if pos > 0.98:
    reward += 20
elif pos > 0.92:
    reward += 0
elif pos > 0.82:
    reward += 6
elif pos > 0.65:
    reward += 1 - np.exp(-2 * pos)

initial_position = 0.40842572

if vel > 0.3 and pos > initial_position + 0.1:
    reward += 1 + 2 * pos

return reward

def train_mountain_car(self):
    if self.env is None:
        raise ValueError("Env is none")

    self.reward_history = []

    with tqdm(range(self.num_episodes), desc="Episode") as prog_bar:
        for episode in prog_bar:
            state, _ = self.env.reset(seed=self.seed)
            episode_reward: float = 0.0
            episode_loss: float = 0.0

            for step in range(self.num_steps):
                norm_state = self.normalize_state(state, self.env)
                action = self.agent.select_action(state)
                next_state, reward, done, truncated, _ = self.env.step(action)

                # modify reward if it's mountain car
                if self.modify_reward:
                    mod_reward = self.modify_reward(next_state)
                else:
                    mod_reward = reward

                self.agent.replay_buffer.push(
                    norm_state,
                    action,
                    mod_reward,
                    self.normalize_state(next_state, self.env),
                    done,
                )

                self.agent.train()

            episode_reward += float(mod_reward)
            state = next_state

            episode_loss += float(self.agent.loss)

            if step % self.update_target_frequency == 0:

```



```

        self.agent.update_target()

        if done:
            break

        self.agent.update_epsilon()

        self.reward_history.append(episode_reward)
        self.loss_history.append(episode_loss)

        # show mean reward for last 10 episodes
        avg_reward = np.mean(self.reward_history[-10:])
        prog_bar.set_postfix(
            {
                "avg_reward": f"{avg_reward:.2f}",
                "epsilon": f"{self.agent.epsilon:.3f}",
            }
        )

def train_lunar_lander(self):
    for episode in tqdm(range(self.num_episodes), desc="Episode"):
        state, _ = self.env.reset(seed=self.seed)
        episode_reward: float = 0.0
        episode_loss: float = 0.0

        for step in range(self.num_steps):
            action = self.agent.select_action(state)
            next_state, reward, done, truncated, info = self.env.step(action)
            self.agent.replay_buffer.push(state, action, reward, next_state, done)

            state = next_state
            episode_reward += float(reward)

            self.agent.train()
            episode_loss += float(self.agent.loss)

            if done:
                break

        self.reward_history.append(episode_reward)
        self.loss_history.append(episode_loss)

    self.agent.update_target()

def run(self, env_name):
    if env_name == "MountainCar-v0":
        self.train_mountain_car()
    elif env_name == "LunarLander-v3":
        self.train_lunar_lander()
    else:
        raise ValueError("Simulation.run: Wrong env name.")

def get_env(self, env_name):
    """
    Returns env, obs_size, n_actions, params (agent and some simulation parameters)
    """
    env = None
    n_actions = None
    obs_size = None
    params = None

    if env_name == "LunarLander-v3":

```

```

env = gym.make(
    "LunarLander-v3",
    continuous=False,
    gravity=-10.0,
    enable_wind=False,
    wind_power=15.0,
    turbulence_power=1.5,
)
n_actions = 4
obs_size = 8
params = {
    "layers": lambda obs_size, n_actions: [
        nn.Linear(obs_size, 256),
        nn.ReLU(),
        nn.Linear(256, 128),
        nn.ReLU(),
        nn.Linear(128, 64),
        nn.ReLU(),
        nn.Linear(64, n_actions),
    ],
    "gamma": 0.99,
    "epsilon": 0.9,
    "epsilon_decay": 0.955,
    "epsilon_min": 0.05,
    "batch_size": 64,
    "num_steps": 300,
    "num_episodes": 1000,
    "lr": 3e-2,
    "target_update_frequency": 10,
    "modify_rewards": False,
}
elif env_name == "MountainCar-v0":
    env = gym.make(
        "MountainCar-v0",
        max_episode_steps=200,
        render_mode="rgb_array",
        goal_velocity=0.1,
    )
    n_actions = 3
    obs_size = 2
    params = {
        "layers": lambda obs_size, n_actions: [
            nn.Linear(obs_size, 12),
            nn.ReLU(),
            nn.Linear(12, 8),
            nn.ReLU(),
            nn.Linear(8, n_actions),
        ],
        "gamma": 1.0,
        "epsilon": 0.999,
        "epsilon_decay": 0.997,
        "epsilon_min": 0.01,
        "batch_size": 64,
        "num_steps": 200,
        "num_episodes": 1000,
        "lr": 75e-5,
        "target_update_frequency": 10,
        "modify_rewards": True,
    }
else:
    raise ValueError("Wrong env name.")

return env, obs_size, n_actions, params

```

```

if __name__ == "__main__":
    env = None
    actions = None
    obs = None

    env_names = [
        "MountainCar-v0",
        "LunarLander-v3",
    ]

    experiments = [
        ("dqn", default_experiment),
        ("double_q", default_experiment),
        ("prioritized", default_experiment),
        ("dqn", dueling_experiment),
    ]

    for env_name in env_names:
        for agent_algorithm, experiment in experiments:
            env, obs, actions, params = get_env(env_name)
            experiment(
                env, obs, actions, params, env_name, agent_algorithm=agent_algorithm
            )

```